

자료구조 실습 보고서

[제 08 주] 수식 계산

제출일 : 2018 년 5 월 1 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘

- `public boolean infixToPostfix() : infix 를 postfix 로 변환`

infix 를 차례대로 순회하면서, 각 인덱스에 들어있는 (토큰)을 처리하며 postfix 의 형태를 만든다.
토큰이 숫자이면, 그대로 postfix 에 추가한다.

토큰이 ')' 이면 연산자 스택인 oStack 에서 '('가 나올 때까지 계속 pop 하여 postfix 에 추가한다.

토큰이 '(' 또는 다른 연산자이면 oStack 에 넣는데, 넣으려는 연산자보다 더 우선순위가 높은 것이 이미 들어있다면 그것들을 모두 pop 하여 postfix 에 붙여준다. 이는 우선순위가 더 높은 연산들을 먼저 하기 위함이다.

상기 모든 과정이 끝나고 나면, 현재 연산자 스택 oStack 에 남아있는 연산자들을 모두 pop 하여 postfix 에 추가한다.

이 과정들을 전부 마치면, postfix 로의 변환이 완료된다.

ex) infix : (1 + 2) / 3

Token : '('	postfix :	OStack : (
Token : '1'	postfix : 1	OStack : (
Token : '+'	postfix : 1	OStack : (+
Token : '2'	postfix : 1 2	OStack : (+
Token : ')'	postfix : 1 2 +	OStack :
Token : '/'	postfix : 1 2 +	OStack : /
Token : '3'	postfix : 1 2 + 3	OStack : /
pop	postfix : 1 2 + 3 /	OStack :

- `public double evalPostfix() : postfix 로 변환된 식을 계산`

infixToPostfix 로 만들어진 식은 문자열 형태이다. 따라서, 이를 계산하기 이전에 숫자로 변환하는 과정이 필요하다.

기본적으로 postfix 는 왼쪽에서부터 계산하게 되는데, 요소를 탐색해가며 숫자를 만나면

값을 저장하는 스택인 vStack 에 push 하고 연산자를 만나면 vStack 에서 값을 2 개 빼내어 해당 연산을 수행한다.

이를 식의 끝을 나타내는 '\$' 가 나올 때까지 반복하면, vStack 에는 계산의 결과만이 남아있게 된다.

b. 함수 설명서

■ AppController

- `public void run() : 프로그램 실행`

- public void evalExpression() : _calculator 에 저장된 infix 식을 postfix 로 바꾼 후, 계산하여 값을 출력하는 과정을 명령

■ AppView

- public String inputString() : 문자열 입력받음
- public String inputExpression() : 수식 입력받음
- public void outputResult(double aValue) : 계산한 결과를 받아 출력
- public void outputPostfix(String aPostfix) : Postfix 를 받아 출력
- public void outputMessage(String aMessage) : 문자열 출력

■ ArrayList

- public ArrayList() : top 초기화, capacity 를 기본값인 5 로 초기화, capacity 만큼 배열 할당
- public ArrayList(int givenCapacity) : top 초기화, capacity 를 givenCapacity 로 초기화, capacity 만큼 배열 할당
- public boolean isEmpty() : 비었는 지 여부 반환
- public boolean isFull() : 다 찼는지 여부 반환
- public int size() : 현재 스택의 크기 출력
- public boolean push(T anElement) : 스택에 원소 추가
- public T pop() : 스택에서 원소 제거 후 반환
- public T peek() : 스택의 Top 위치의 원소 반환
- public void clear() : 배열의 원소 전부 null 로 초기화
- public T elementAt(int anOrder) : 인자로 받은 anOrder 의 위치가 유효할 시 해당 위치의 원소 반환, 유효하지 않을 시 null 반환

■ Calculate

- public void setInfix(String newInfix) : infix 설정 (Setter)
- public String infix() : infix 반환 (Getter)
- public boolean infixToPostfix() : 필드에 저장된 infix 를 Postfix 로 변환
- public String postfix() : infixToPostfix()를 이용하여 생성이 완료된 postfix 반환 (Getter)
- public double evalPostfix() : infixToPostfix()를 이용하여 생성이 완료된 postfix 를 계산

c. 종합 설명서

스택을 이용하여, 흔히 사용하는 infix 수식을 컴퓨터가 사용하는 postfix 수식으로 바꾸고 계산하는 프로그램이다.

+, -, /, *, %, ^ 연산을 할 수 있으며 입력할 때에는 공백이 들어가면 안되고 !를 입력하면 종료한다.

2. 프로그램 장단점 분석

장점 : x

단점 : 숫자와 연산자 사이에 공백을 넣으면 에러가 발생한다.

3. 실행 결과 분석

a. 입력과 출력

:: 프로그램을 시작합니다. ::

[수식 입력을 시작합니다.]

> 수식을 입력하십시오 : $((5+3)*9/4\%3-1)^2$

[Infix를 Postfix로]

OStack : (

OStack : ((

OStack : ((+

OStack : (

OStack : (*

OStack : (/

OStack : (%

OStack : (-

OStack :

OStack : ^

[Postfix] : 53+9*4/3%1-2^

VStack : 5.0

VStack : 5.0 3.0

VStack : 8.0

VStack : 8.0 9.0

VStack : 72.0

VStack : 72.0 4.0

VStack : 18.0

VStack : 18.0 3.0

VStack : 0.0

VStack : 0.0 1.0

VStack : -1.0

VStack : -1.0 2.0

VStack : 1.0

[최종값] : 1.0

> 수식을 입력하십시오 : !

[수식 입력을 종료합니다.]

:: 프로그램을 종료합니다. ::

b. 결과분석

입력한 식은 괄호를 포함하기 때문에, 원래 infix 식 계산대로라면 순서가 일정하지 않을 것이다.

하지만 postfix 식은 왼쪽에서부터 차례대로 계산해나가면 되며 괄호가 필요하지 않다.

변환된 postfix 식과 입력한 식과의 관계를 확인하면 될 것이다.

입력한 식은 $((5+3)*9/4\%3-1)^2 = (8*9/4\%3-1)^2 = (72/4\%3-1)^2 = (18\%3-1)^2 = (0-1)^2 = (-1)^2 = 1$

Postfix 의 계산 과정을 보면,

1. 5 3+ : 8

2. 8 9* : 72

3. 72 4/ : 18

4. 18 3 % : 0

5. 0 1 - : -1

6. -1 2 ^ : 1

따라서 계산 과정과 답이 들어맞으므로 맞는 결과라고 할 수 있다.

4. 소스코드

AppController.java

```
public class AppController {

    private AppView _appView;
    private Calculate _calculate;

    // 생성자
    public AppController() {
        this._appView = new AppView();
        this._calculate = new Calculate();
    }

    // infix를 postfix로 변환하여 출력, 계산
    public void evalExpression() {

        double finalValue;
        this.showMessage(MessageID.Notice_InfixToPostfix);

        if(this._calculate.infixToPostfix()) {
            this._appView.outputPostfix(this._calculate.postfix());
            finalValue = this._calculate.evalPostfix();
            this._appView.outputResult(finalValue);
        }
        else {
            this.showMessage(MessageID.Error_Input);
        }
    }

    public void run() {
        this.showMessage(MessageID.Notice_StartProgram);
        this.showMessage(MessageID.Notice_StartMenu);

        String input = this._appView.inputExpression();
        while(input.charAt(0) != '!') {
            this._calculate.setInfix(input);
            this.evalExpression();
            input = this._appView.inputExpression();
        }
        this.showMessage(MessageID.Notice_EndMenu);
        this.showMessage(MessageID.Notice_EndProgram);
    }

    private void showMessage(MessageID aMessageID) {

        switch(aMessageID) {

            case Notice_StartProgram:
                this._appView.outputMessage(" :: 프로그램을 시작합니다. :: \n");
                break;
            case Notice_EndProgram:
                this._appView.outputMessage(" :: 프로그램을 종료합니다. :: \n");
                break;
            case Notice_StartMenu:
                this._appView.outputMessage("\n[ 수식 입력을 시작합니다. ] \n");
                break;
        }
    }
}
```

```

        case Notice_EndMenu:
            this._appView.outputMessage("\n[ 수식 입력을 종료합니다. ] \n");
            break;
        case Notice_InfixToPostfix:
            this._appView.outputMessage("\n[ Infix를 Postfix로 ] \n");
            break;
        case Error_Input:
            this._appView.outputMessage("! 잘못된 입력 \n");
            break;
    }
}
}

```

AppView.java

```

import java.util.Scanner;

public class AppView {

    private Scanner _scanner;

    // 생성자
    public AppView() {
        this._scanner = new Scanner(System.in);
    }

    // 입력

    // 문자열 입력
    public String inputString() {
        return this._scanner.nextLine();
    }

    // 수식 입력
    public String inputExpression() {
        System.out.print("> 수식을 입력하시오 : ");
        return this.inputString();
    }

    // 출력

    // 최종 값 출력
    public void outputResult(double aValue) {
        System.out.println("\n[최종값] : " + aValue);
    }

    // Postfix 출력
    public void outputPostfix(String aPostfix) {
        System.out.println("\n[Postfix] : " + aPostfix);
    }

    // 문자열 출력
    public void outputMessage(String aMessage) {
        System.out.print(aMessage);
    }
}

```

ArrayList.java

```

public class ArrayList<T> implements Stack<T> {

    private static final int DEFAULT_CAPACITY = 5;
    private int _capacity;
    private int _top;
    private T[] _elements;

    // 생성자
    @SuppressWarnings("unchecked")
    public ArrayList() {
        this._capacity = ArrayList.DEFAULT_CAPACITY;
        this._top = -1;
        this._elements = (T[])new Object[ArrayList.DEFAULT_CAPACITY];
    }

    @SuppressWarnings("unchecked")
    public ArrayList(int givenCapacity) {
        this._capacity = givenCapacity;
        this._top = -1;
        this._elements = (T[])new Object[this._capacity];
    }

    // 공개 함수

    // 비어있는지 확인
    public boolean isEmpty() {
        return (this._top == -1);
    }

    // 꽉 찼는지 확인
    public boolean isFull() {
        return (this.size() == this._capacity);
    }

    // 총 길이 확인 (top의 인덱스 번호를 통함)
    public int size() {
        return (this._top + 1);
    }

    // 원소 추가 (Top)
    public boolean push(T anElement) {
        if(this.isFull()) {
            return false;
        }
        else {
            this._top++;
            this._elements[this._top] = anElement;
            return true;
        }
    }

    // 원소 제거 (Top)
    public T pop() {
        if(this.isEmpty()) {
            return null;
        }
        else {
            this._top--;
            return this._elements[this._top + 1]; // 삭제된 원소 반환
        }
    }

    // Top 의 원소 확인

```

```

    public T peek() {
        if(this.isEmpty()) {
            return null;
        }
        else {
            return this._elements[this._top];
        }
    }

    // Stack 초기화
    public void clear() {
        this._top = -1;
        for(int index = 0; index < this.size(); index++) {
            this._elements[index] = null;
        }
    }

    // 주어진 순서의 원소 반환
    public T elementAt(int anOrder) {
        if(anOrder >= 0 && anOrder < this.size()) {
            return this._elements[anOrder];
        }
        else {
            return null;
        }
    }
}

```

Stack.java

```

public interface Stack<T> {

    public boolean isEmpty();           // 비어있는지 확인
    public boolean isFull();           // 꽉 찼는지 확인
    public boolean push(T anElement);  // 추가
    public T pop();                     // 제거
    public T peek();                   // Top 의 원소 확인
}

```

Calculate.java

```

public class Calculate {

    private static final char END_OF_TOKEN = '$';

    private ArrayList<Character> _oStack; // 연산자 스택
    private ArrayList<Double> _vStack;   // 값 스택

    private char[] _infix;
    private char[] _postfix;

    // 생성자
    public Calculate() {

    }

    // Setter : infix 재설정
    public void setInfix(String newInfix) {

```



```

        this._infix = newInfix.toCharArray();
    }

    // Getter
    public String infix() {
        return String.valueOf(this._infix);
    }

    public String postfix() {
        // postfix로의 변환이 먼저 되어야 함

        // 괄호의 개수 세기
        int gCount = 0;
        for(int i = 0; i < this._infix.length; i++) {
            if(this._infix[i] == '(' || this._infix[i] == ')') {
                gCount++;
            }
        }

        // 끝에 $를 제거하고 반환
        return String.valueOf(this._postfix).substring(0, this._postfix.length - gCount - 1);
    }

    // infix를 postfix로 변환
    public boolean infixToPostfix() {

        int infixIndex = 0; // string pointer for infix
        int postfixIndex = 0; // string pointer for postfix
        char currentToken, poppedToken, topToken;

        this._oStack = new ArrayList<Character>(this._infix.length);
        this._postfix = new char[this._infix.length + 1];

        // infix 전체 순회
        while(infixIndex < this._infix.length) {

            // infix에서 검사할 토큰 하나 뽑음
            currentToken = this._infix[infixIndex++];

            // currentToken이 숫자 : 바로 postfix에 추가
            if(this.isDigit(currentToken)) {
                this._postfix[postfixIndex++] = currentToken;
            }

            else { // currentToken이 연산자

                // currentToken이 ')'
                if(currentToken == ')') {
                    if(!this._oStack.isEmpty()) {
                        poppedToken = (char)this._oStack.pop();
                    }
                    else {
                        return false;
                    }
                }

                // '(' 까지 계속 pop하면서 postfix에 붙임
                while(poppedToken != '(') {
                    this._postfix[postfixIndex++] = poppedToken;
                    if(!this._oStack.isEmpty()) {
                        poppedToken = (char)this._oStack.pop();
                    }
                    else {

```

postfix에 붙임

```
        return false;
    }
}
this.showOStackAll();
}

// currentToken이 '(' 또는 일반 연산자
else {
    int inComingP = this.inComingPrecedence(currentToken);

    if(!this._oStack.isEmpty()) {
        topToken = (char)this._oStack.peek();

        // 스택 안에 우선 순위가 더 높은 연산자가 있을 시, 전부 pop하면서

        while(this.inStackPrecedence(topToken) >= inComingP) {
            poppedToken = (char)this._oStack.pop();
            this._postfix[postfixIndex++] = poppedToken;

            if(!this._oStack.isEmpty()) {
                topToken = (char)this._oStack.peek();
            }
            else {
                break;
            }
        }
        this._oStack.push(currentToken);
        this.showOStackAll();
    }
}

while(!this._oStack.isEmpty()) {
    this._postfix[postfixIndex++] = this._oStack.pop();
}
this._postfix[postfixIndex] = Calculate.END_OF_TOKEN;

return true;
}
```

push

```
// 변환된 Postfix 수식 계산
public double evalPostfix() {

    int stringIndex = 0;
    char token;
    this._vStack = new ArrayList<Double>(this._infix.length); // postfix 식을 double로 변환하여 여기다

    token = this._postfix[stringIndex++];
    while(token != Calculate.END_OF_TOKEN) {

        // token이 숫자일 때
        if(this.isDigit(token)) {
            double doubleToPush = Double.parseDouble(String.valueOf(token));
            this._vStack.push(doubleToPush);
            this.showVStackAll();
        }

        // token이 operator일 때
        else {
            double secondOperand = this._vStack.pop();
            double firstOperand = this._vStack.pop();
```

```

        switch(token) {
        case '+':
            this._vStack.push(firstOperand + secondOperand);
            break;

        case '-':
            this._vStack.push(firstOperand - secondOperand);
            break;

        case '*':
            this._vStack.push(firstOperand * secondOperand);
            break;

        case '/':
            this._vStack.push(firstOperand / secondOperand);
            break;

        case '%':
            this._vStack.push(firstOperand % secondOperand);
            break;

        case '^':
            this._vStack.push(Math.pow(firstOperand, secondOperand));
            break;
        }
        this.showVStackAll();
    }

    token = this._postfix[stringIndex++];
}

return this._vStack.pop();
}

```

// 비공개 함수

// 문자가 숫자 문자인지 확인

```

private boolean isDigit(char aToken) {
    return (aToken >= '0' && aToken <= '9');
}

```

// 현재 넣을 연산자인 Token의 우선순위 값 반환

```

private int inComingPrecedence(char aToken) {
    if(aToken == '+')
        return 12;
    else if(aToken == '-')
        return 12;
    else if(aToken == '(')
        return 20;
    else if(aToken == ')')
        return 19;
    else if(aToken == '*')
        return 13;
    else if(aToken == '/')
        return 13;
    else if(aToken == '%')
        return 13;
    else if(aToken == '^')
        return 17;
    else if(aToken == '$')
        return 0;
    else

```

```

        return -1;
    }

    // 스택 안의 연산자 Token의 우선순위 값 반환
    private int inStackPrecedence(char aToken) {
        if(aToken == '+')
            return 12;
        else if(aToken == '-')
            return 12;
        else if(aToken == '(')
            return 0;
        else if(aToken == ')')
            return 19;
        else if(aToken == '*')
            return 13;
        else if(aToken == '/')
            return 13;
        else if(aToken == '%')
            return 13;
        else if(aToken == '^')
            return 16;
        else if(aToken == '$')
            return 0;
        else
            return -1;
    }

    // Operation 스택의 상태 확인
    private void showOStackAll() {
        System.out.print("OStack : ");
        for(int index = 0; index < this._oStack.size(); index++) {
            System.out.print(this._oStack.elementAt(index) + " ");
        }
        System.out.println();
    }

    // Value 스택의 상태 확인
    private void showVStackAll() {
        System.out.print("VStack : ");
        for(int index = 0; index < this._vStack.size(); index++) {
            System.out.print(this._vStack.elementAt(index) + " ");
        }
        System.out.println();
    }
}

```

MessageID.java

```

public enum MessageID {

    // Message IDs for Notices
    Notice_StartProgram,
    Notice_EndProgram,

    Notice_StartMenu,
    Notice_EndMenu,

    Notice_InfixToPostfix,

    // Message IDs for Errors
    Error_Input

}

```

DS1_08_201702081_최재범.java

```
public class DS1_08_201702081_최재범 {  
  
    public static void main(String[] args) {  
        AppController appController = new AppController();  
        appController.run();  
    }  
  
}
```